Task Force or the IETF) summarises this feature crisply: "HTTP/2 allows a server to pre-emptively send (or 'push') responses to a client in association with a previous client-initiated request." This can be useful when the server knows the client will need to have those responses available in order to fully process the response to the original request. Thus, when a client sends a single request in a stream, it is possible that it receives one or more responses, of which the first is the direct response, say R1. The other (optional) responses, say R2 and R3, could be in anticipation of requests that the client may make based on processing R1. If the client indeed followed that path and required responses for R2 and/or R3, it is saved from making those requests.

Header compression: As stated earlier, each HTTP/2 stream roughly corresponds to the HTTP 1.1 request/response pair. While no semantic changes have been made in streams, the format of streams has been optimised. The HTTP headers, which were in text in HTTP 1.1, have been subject to compression and thus converted to a binary format.

The overall compression approach and algorithms make up a topic that can be discussed further and even published as a different RFC. We limit ourselves to a brief overview. The compression scheme is stateful, which implies that both the endpoints must have the right initial state to successfully compress and decompress the stream headers. This is a per-connection state and is shared for all streams on that connection.

HTTP/2: Miscellany

HTTP/2 is a wide specification and it is not possible to cover all its facets here. However, we briefly mention some more aspects, so that the interested reader can pursue them further.

1. **Stream**
   a. *State machine*: Each stream follows a state machine that indicates the messages that a stream can send/receive in those states and the error conditions thereof.
   b. *Priority:* It is possible for either endpoint to indicate which stream(s) is(are) of higher priority. This is, for example, useful for the peer to make decisions about sending traffic during phases of resource limitations.
2. **Frames**
   a. *SETTINGS:* As alluded to earlier, the SETTINGS frame allows an endpoint to communicate the limits of various protocol entities like the maximum number of concurrent streams, initial window sizes, sizes of the header compression table, maximum frame size, and so on.
   b. *HEADERS:* A stream is opened with the HEADERS frame, whose payload contains the compressed headers.
   c. *CONTINUATION:* When the HEADERS do not fit within the maximum frame size supported by the peer, then the sender breaks them into multiple frames. All these frames, other than the first one, are marked as CONTINUATION of the HEADERS frame.
3. **Error handling:** Any network communication requires propagating errors to the remote end as soon as they are detected. In this perspective, HTTP 2.0 supports two kinds of errors — connection and stream errors. Some examples of these are:

a. *Connection errors:* Decoding error in a header block, decompression errors with a header block, certain kinds of stream state machine violations, unexpected stream identifiers, and so on.

b. *Stream errors:* Receiving headers on a stream that causes stream limits to be exceeded, certain other kinds of stream state machine violations, PRIORITY stream of greater than 5 bytes, and so on.

TLS 1.2

The SSL protocol was first defined by Netscape and it was an excellent example of tying together the concepts of asymmetric and symmetric cryptography along with PKI to provide end-to-end security for entities that did not have any prior interaction. This was just what was needed on the Internet, where clients and servers interact without prior off-band communication.

While the protocol was very successful, crypto experts also observed the difficulties in building an open end-to-end secure protocol. We refer to the protocol as open because of the following reasons:

1. The complete specification of the protocol is available to attackers
2. Attackers have access to both client and server implementations

Note that being an open protocol is very desirable from a security standpoint, as it abides by the following law proposed by the Dutch cryptographer, Auguste Kerchoffs, "A cryptosystem should be secure even if everything about the system, except the key, is public knowledge."

Getting back to the topic, the SSL protocol had two published versions — SSLv2 and SSLv3, before it was submitted to the IETF and standardised as TLS 1.0. While the engineering community was satisfied with SSL, the cryptography experts were aware of limitations in the protocol and hence, worked to define the next version to overcome some of them. TLS 1.2 was proposed and standardised a few years before the security researchers began identifying vulnerabilities in SSLv3. POODLE and BEAST were two of the important vulnerabilities found in SSLv3. However, by the time they were discovered, not only TLS 1.0 and TLS 1.1 but even TLS 1.2 was well under deployment. So, the Internet community was prepared to deal with SSLv3 vulnerabilities and in fact, these helped in accelerating the adoption of TLS 1.2.